



LuxuryStay Hospitality – MERN Stack Project

MERN Hotel Management System Documentation

TABLE OF CONTENTS

Preliminary Pages

1. Certificate	1
2. Dedication	2
3. Acknowledgements	3
4. Abstract	4
5. Table of Contents	5
6. List of Tables	8
7. List of Figures	9

1. Chapter 1 - Introduction

1.1 Introduction	7
1.2 Problem Statement	10
1.3 Objectives	12
1.4 Challenges	13
1.5 Limitations	14
1.6 Project Scope	15
1.7 Functional Requirements	16

2. Chapter 2 - Literature Survey

2.1 Existing Hotel Management Systems	17
2.2 MERN Stack Advantages	18
2.3 Gaps in Current Solutions	19

3. Chapter 3 - System Design

3.1 System Architecture	20
3.2 Use Case Diagram	22
3.3 Data Flow Diagram	23
3.4 Activity Diagram	24

LuxuryStay Hospitality – MERN Stack Project

3.5 ER Diagram	25
3.6 Sequence Diagram	26
3.7 Class Diagram	27
3.8 Gantt Chart	28

4. Chapter 4 - Implementation

4.1 Technology Stack	29
4.2 Frontend Development	30
4.3 Backend Development	31
4.4 Database Integration	32
4.5 Testing Strategy	33

5. Chapter 5 - Results and Discussion

5.1 Source Code with Comments	34
5.2 Performance Analysis	35
5.3 User Feedback	36

6. Chapter 6 - Conclusion and Future Work

6.1 Conclusion	37
6.2 Future Enhancements	38

References	39
------------------	----

Appendices

1. eProject Synopsis
 2. eProject Analysis
 3. eProject Design
 4. Flowcharts
 5. Process Diagrams
 6. Database Design / Structure
 7. User Guide
 8. Developer's Guide
 9. Module Descriptions
-

LuxuryStay Hospitality – MERN Stack Project

PROJECT TEAM: BATCH -2210F1

Group Members:

- **Student1418744** - MUHAMMAD WASIF KHALIL UR REHMAN
- **Student1421851** - RABIA IMTIAZ
- **Student1418747** - KHALIL ULLAH MUZAMMIL
- **Student1424197** - RAMEEN SAHAR
- **Student1412745** – TAYYABA

PROJECT TIMELINE

Start Date: 12-Feb-2025

End Date: 14-Mar-2025

MERN Stack Project: LuxuryStay Hospitality

CERTIFICATE

This page certifies that the project titled "LuxuryStay Hospitality" has been successfully completed under the guidance of Sir Bilal Fatah.

DEDICATION

Dedicated to Aptech Learning Center for their unwavering support and encouragement. Your guidance has been instrumental in shaping our skills and confidence, providing us with invaluable knowledge and opportunities. We are truly grateful for your dedication and inspiration, which have motivated us to strive for excellence. The learning environment you have created fosters growth, creativity, and a passion for continuous improvement.

ACKNOWLEDGEMENTS

We extend our gratitude to:

- **LuxuryStay Hospitality** for providing the problem statement and requirements.
- Our mentor, **Sir Bilal Fattah** for technical guidance.
- The **eProjects Team** for their structured learning framework.

ABSTRACT

The **MERN Hotel Management System** modernizes operations for LuxuryStay Hospitality by automating reservations, billing, housekeeping, and reporting. Built on the MERN stack (MongoDB, Express.js, React, Node.js), the system enhances efficiency through real-time updates, role-based access. This documentation outlines the system's architecture, design, implementation, and testing phases, demonstrating its scalability and impact on operational workflows.

1. Chapter 1 - Introduction

1.1 Introduction

LuxuryStay Hospitality, a high-end hotel chain, aims to revolutionize its operational efficiency by implementing a modern Hotel Management System (HMS). The system will streamline workflows, enhance guest experiences, and provide insightful data for strategic decision-making.

1.2 Problem Statement

Manual room allocation, billing inaccuracies, and poor inter-department coordination hinder LuxuryStay's growth. The HMS addresses these challenges by automating workflows, providing real-time data access, and ensuring secure, scalable operations.

1.3 Objectives

- Develop a user-friendly system for reservations, check-in/out, and billing.
- Implement role-based access for staff (admin, receptionist, housekeeping).

1.4 Challenges

- Integrating real-time updates for room status.
- Ensuring data security for sensitive guest information.

1.5 Limitations

- IoT integration for smart room controls is not included.
- Limited to web and mobile browsers; no native mobile app.

LuxuryStay Hospitality – MERN Stack Project

1.6 Project Scope

- Modules: User management, reservations, billing, housekeeping, reporting.
- Target Users: Staff, guests, administrators.

1.7 Functional Requirements

Module	Features
User Management	Admin dashboard, guest/staff profiles.
Reservations	Online booking, room availability checks.
Billing	Dynamic pricing, invoice generation.

Chapter 2 - Literature Survey

2.1 Existing Hotel Management Systems

Overview of Traditional Hotel Management Systems

Traditional Hotel Management Systems (HMS) primarily focus on managing reservations, guest check-ins/check-outs, and billing processes. These systems often rely on manual data entry or standalone desktop applications, making them prone to inefficiencies.

Legacy systems typically include:

- **Paper-based systems** – Used by small-scale hotels but are inefficient and error-prone.
- **On-premise software** – Installed on local machines, requiring dedicated maintenance.
- **Basic web-based platforms** – Limited cloud integration and lack advanced automation features.

Challenges Faced by Existing Solutions

Despite their effectiveness in handling basic hotel operations, traditional HMS face several challenges:

- **Limited Scalability** – Most legacy systems struggle to accommodate growing customer bases.

LuxuryStay Hospitality – MERN Stack Project

- **Lack of Real-time Updates** – Manual or semi-automated processes lead to delays in room availability and reservation updates.
 - **Inefficient Customer Experience** – Guests often face delays in bookings and check-in due to outdated interfaces.
 - **Data Security Concerns** – Many older systems do not use encryption, making sensitive customer data vulnerable.
 - **Integration Issues** – Difficulty in integrating with third-party services like online booking platforms, payment gateways, and analytics tools.
-

2.2 MERN Stack Advantages

The **MERN (MongoDB, Express.js, React, Node.js)** stack offers a modern, full-stack JavaScript solution that addresses the shortcomings of traditional HMS.

Benefits of Using the MERN Stack

1. **Scalability** – MongoDB's NoSQL structure allows seamless scaling, accommodating hotel chains of all sizes.
 2. **Real-time Updates** – Node.js enables instant room availability and booking updates.
 3. **Enhanced User Experience** – React.js provides an interactive and dynamic UI for smooth customer interactions.
 4. **Robust Security** – Built-in authentication and authorization mechanisms (JWT, OAuth) ensure data protection.
 5. **Seamless API Integration** – Express.js simplifies the integration of third-party services like payment gateways and online booking platforms.
 6. **Cloud Deployment** – The MERN stack supports cloud hosting, reducing infrastructure costs while ensuring high availability.
-

2.3 Gaps in Current Solutions

Identifying Areas for Improvement

Despite advancements in HMS technology, there are still gaps that modern solutions like a MERN-based HMS can address:

- **AI-driven Room Allocation** – Traditional systems lack smart algorithms to optimize room assignments based on guest preferences.
- **Predictive Maintenance** – Existing solutions fail to provide AI-based maintenance tracking for improved efficiency.
- **Advanced Analytics & Insights** – Many HMS do not offer real-time data analytics for decision-making.

LuxuryStay Hospitality – MERN Stack Project

- **Seamless Mobile Integration** – While many platforms offer mobile support, few provide a fully responsive and feature-rich mobile experience.
- **Community Engagement Features** – No significant platforms enable guest feedback loops and service personalization.

By leveraging the MERN stack, the proposed HMS can introduce these enhancements, improving efficiency, customer satisfaction, and overall hotel management.

3. Chapter 3 - System Design

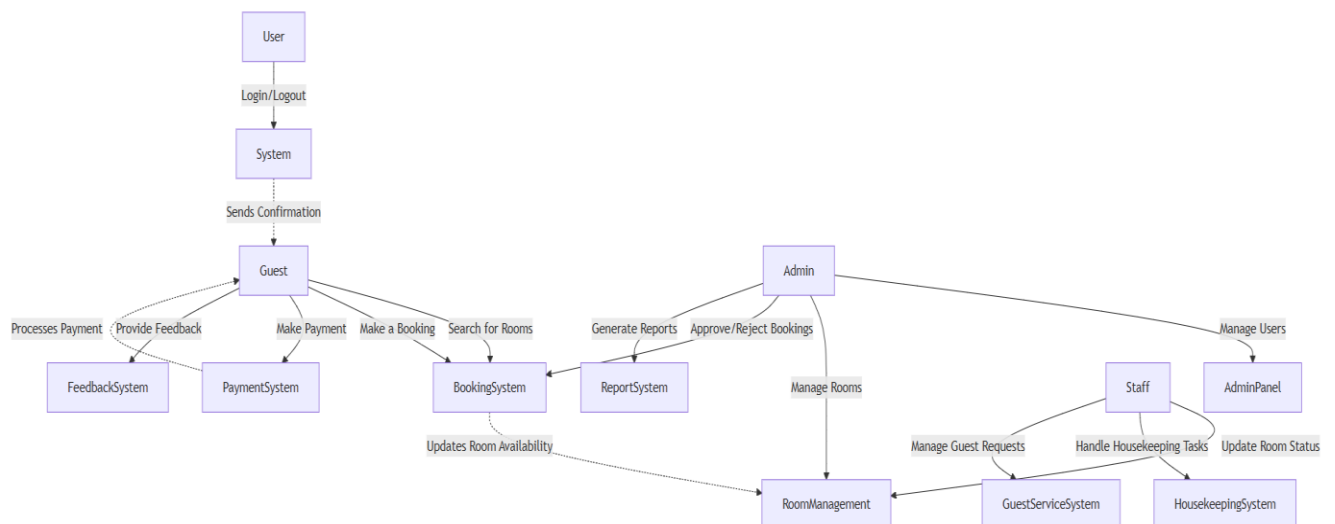
3.1 System Architecture

- **Frontend:** React.js for dynamic UI components (e.g., dashboards).
- **Backend:** RESTful APIs using Node.js and Express.js.
- **Database:** MongoDB for storing guest data, bookings, and invoices.

3.2-3.8 Diagrams

- Use Case Diagram, Data Flow Diagram, Activity Diagram, ER Diagram, Sequence Diagram, Class Diagram, and Gantt Chart.

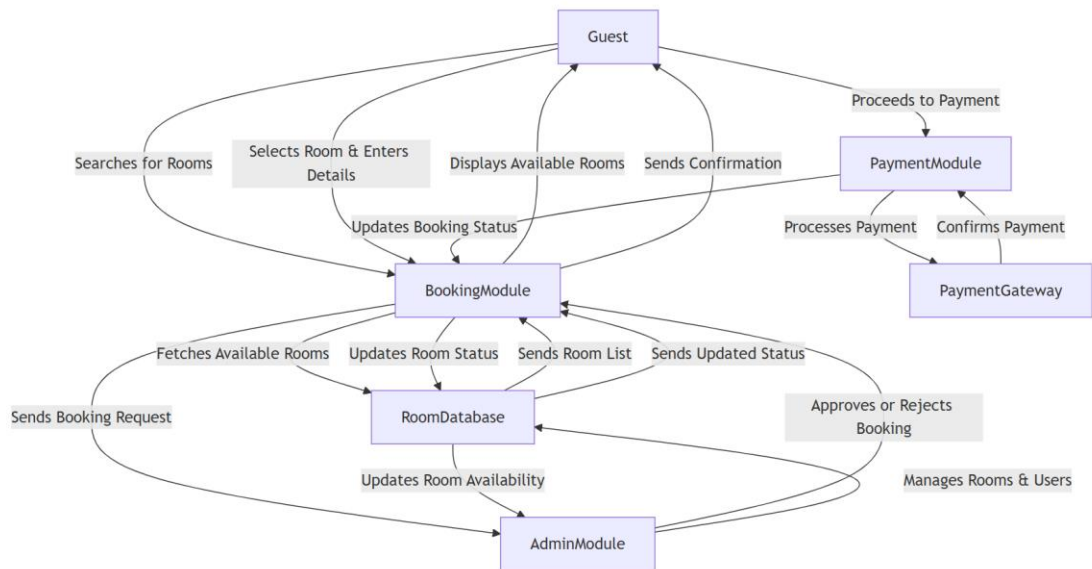
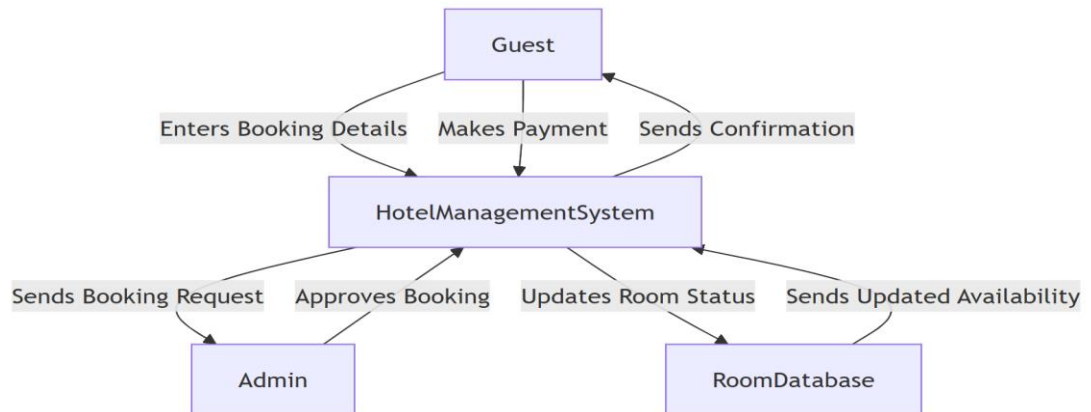
3.2 Use Case Diagram



LuxuryStay Hospitality – MERN Stack Project

3.3 Data Flow Diagram (DFD)

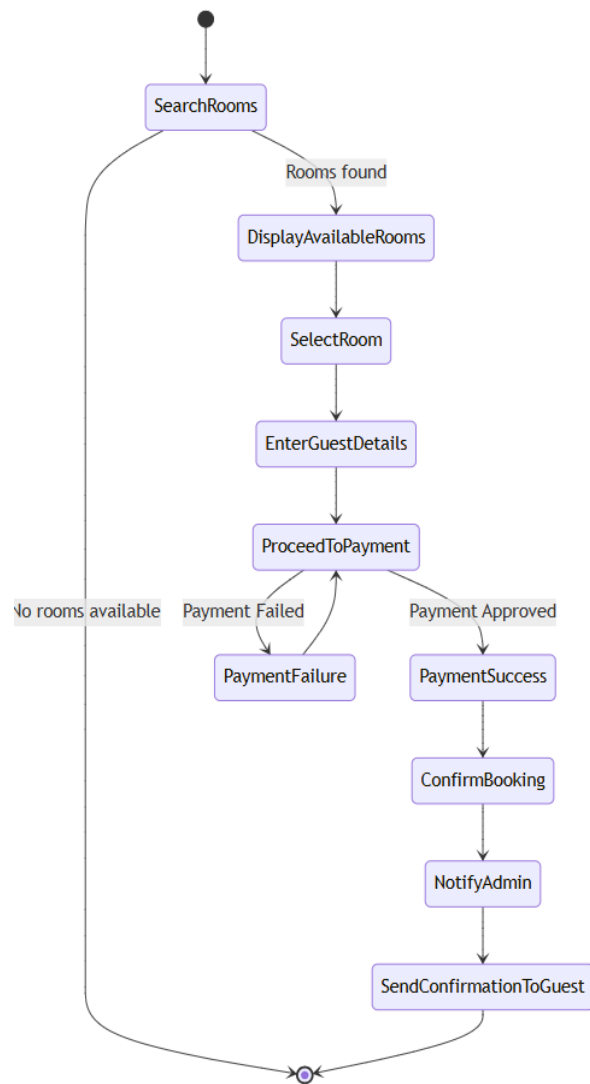
Data Flow diagram for level 0



Data flow diagram for level 1

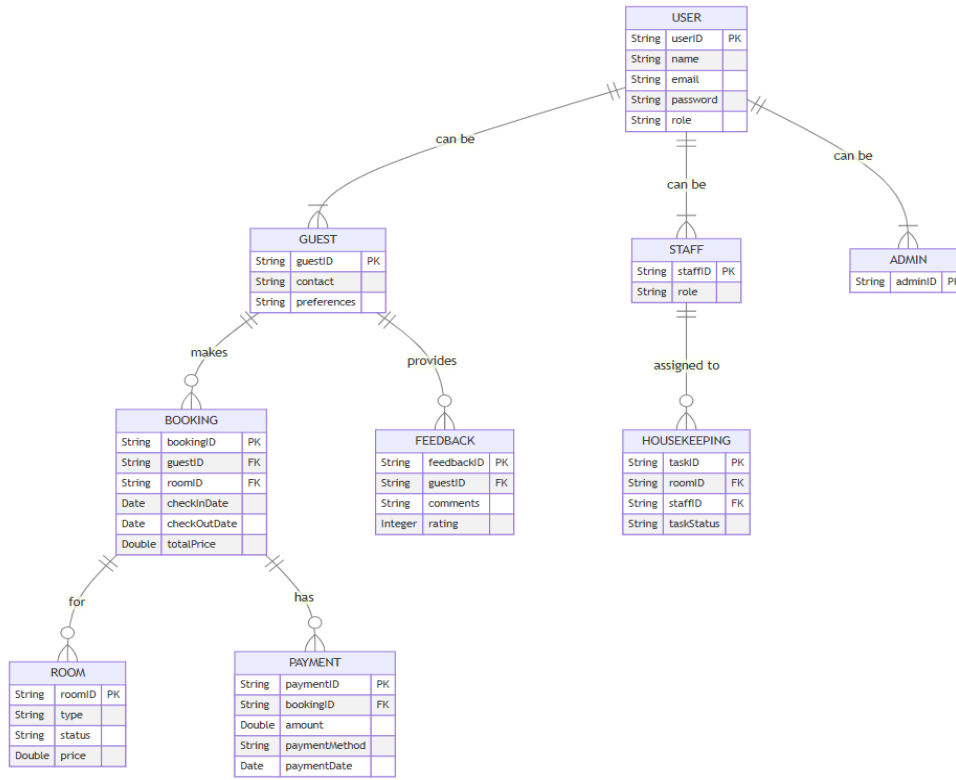
LuxuryStay Hospitality – MERN Stack Project

3.4 Activity Diagram

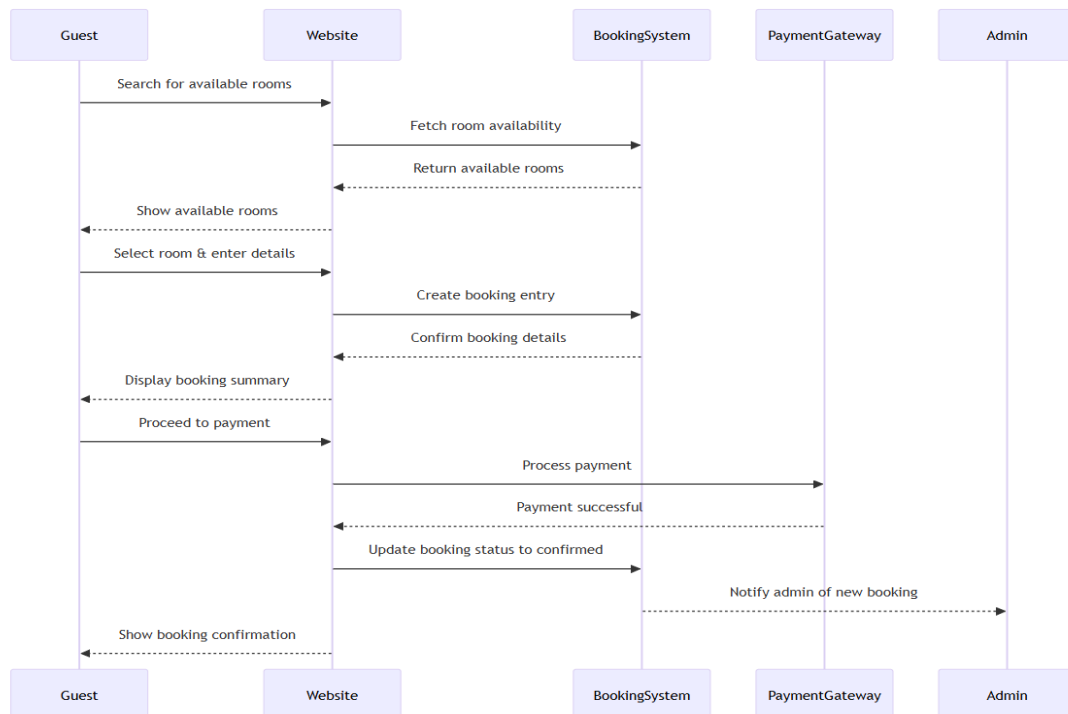


LuxuryStay Hospitality – MERN Stack Project

3.5 ER Diagram

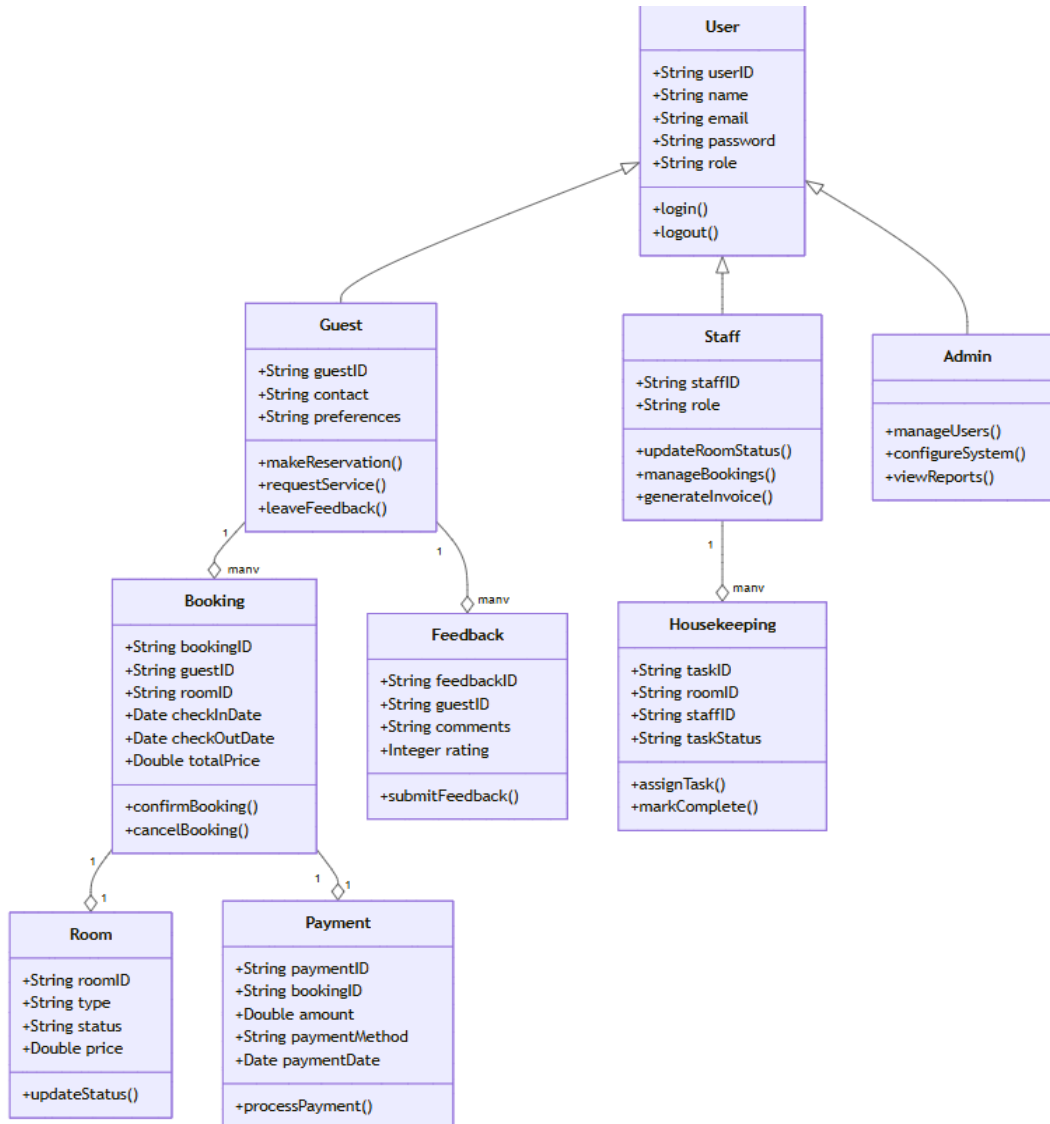


3.6 Sequence Diagram

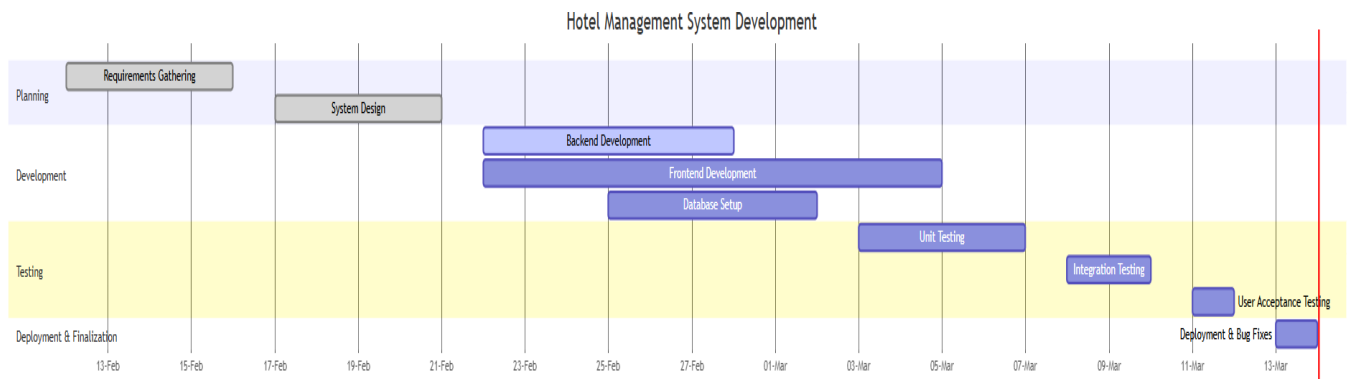


LuxuryStay Hospitality – MERN Stack Project

3.7 Class Diagram



3.8 Gantt Chart



LuxuryStay Hospitality – MERN Stack Project

4. Chapter 4 - Implementation

4.1 Technology Stack

- **Frontend:** React, Axios.
- **Backend:** Node.js, Express.js, JWT.
- **Database:** MongoDB.

4.2 Frontend Development

- Guest portal for bookings.
- Admin dashboard for analytics and user management.

4.3 Backend Development

- REST APIs for CRUD operations on rooms, bookings, and users.
- Integration with Stripe for payments.

4.4 Database Integration

- MongoDB collections: Users, Rooms, Bookings.
 - Schema validation using Mongoose.
Users: {_id, name, role, email, password (hashed)}
 - Rooms: {_id, type, status, price}
 - Bookings: {_id, guestId, roomId, checkIn, checkOut, total}

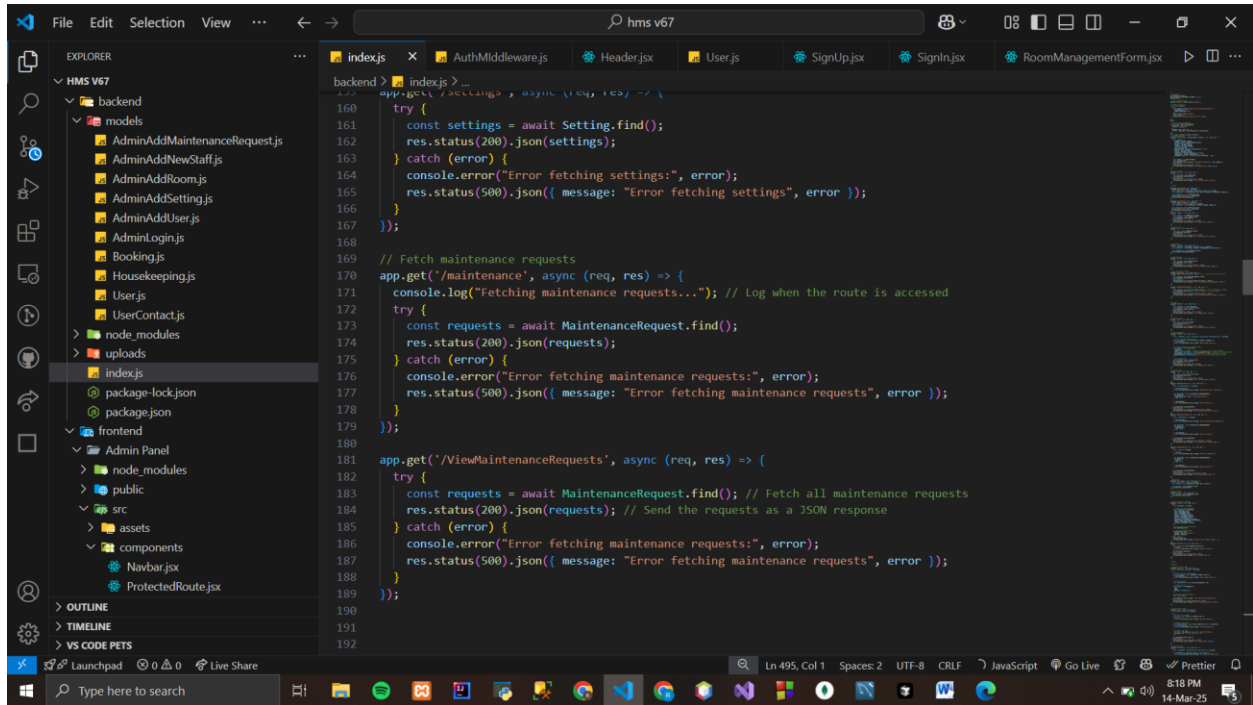
4.5 Testing Strategy

- **Unit Tests:** Jest for API endpoints.
 - **UI Tests:** Cypress for guest booking flow.
-

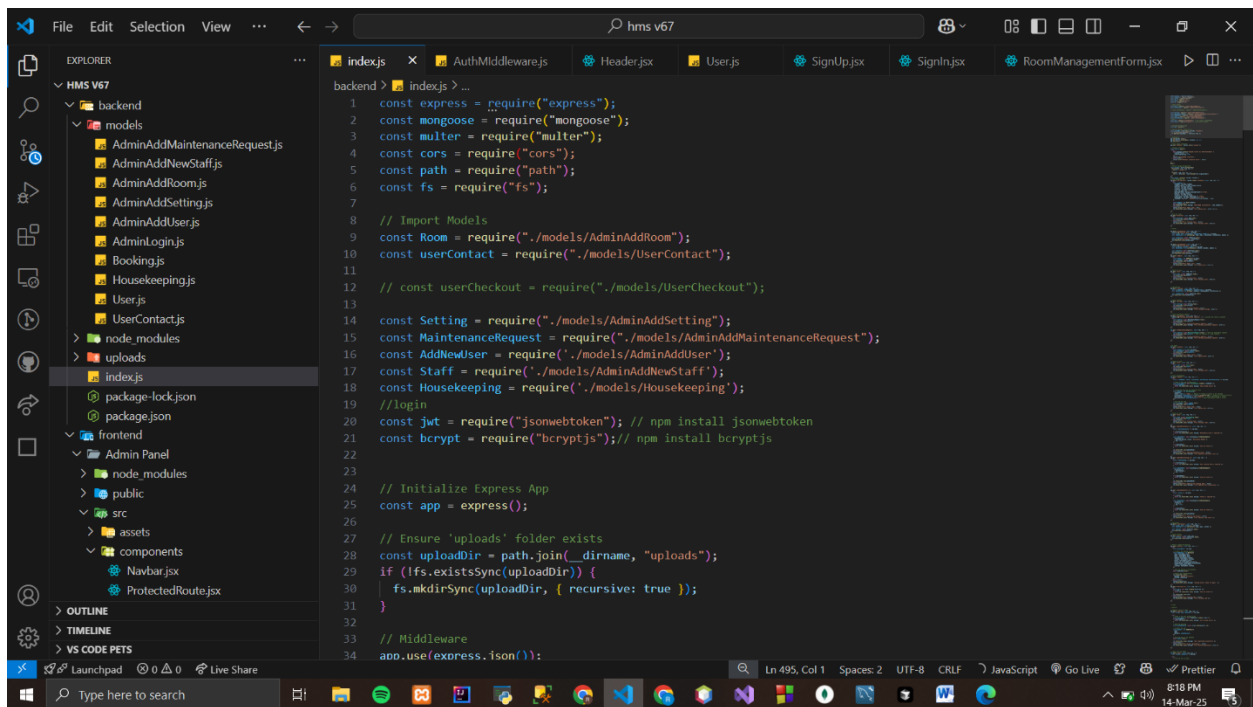
LuxuryStay Hospitality – MERN Stack Project

5. Chapter 5 - Results and Discussion

5.1 Screenshots of Source Code with Comments

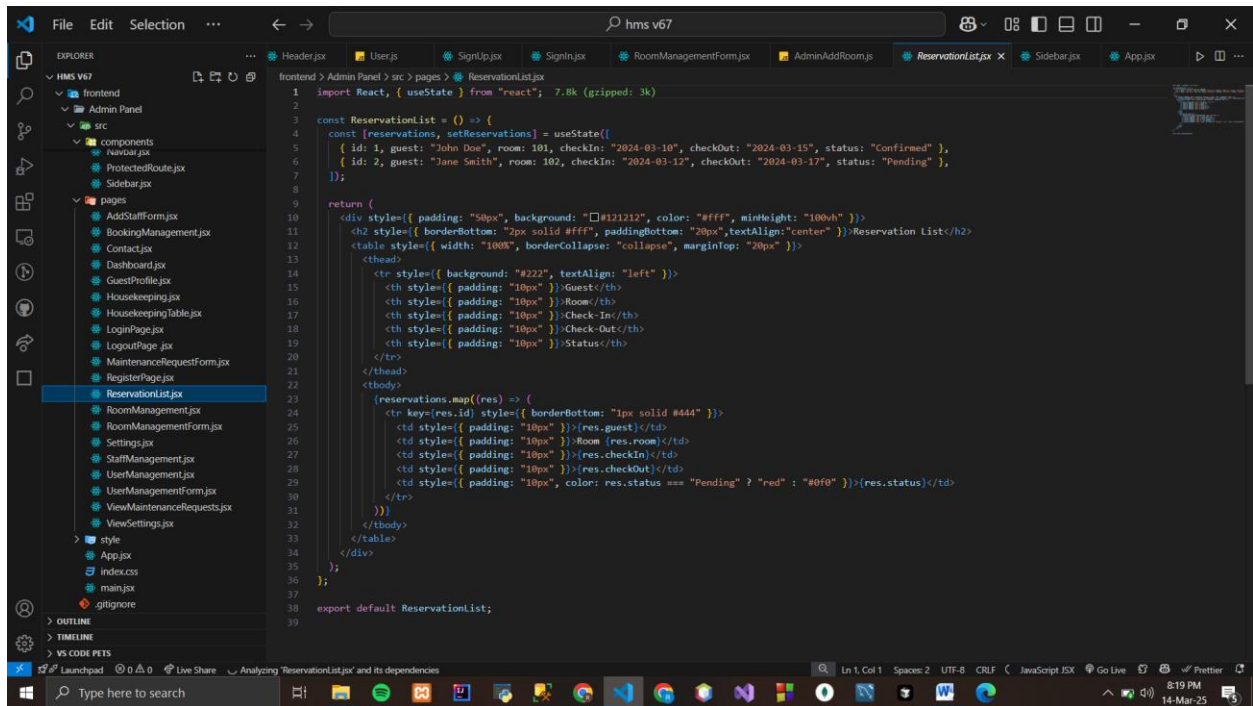


```
160     try {
161       const settings = await Setting.find();
162       res.status(200).json(settings);
163     } catch (error) {
164       console.error("Error fetching settings:", error);
165       res.status(500).json({ message: "Error fetching settings", error });
166     }
167   });
168
169   // Fetch maintenance requests
170   app.get('/maintenance', async (req, res) => {
171     console.log("Fetching maintenance requests..."); // Log when the route is accessed
172     try {
173       const requests = await MaintenanceRequest.find();
174       res.status(200).json(requests);
175     } catch (error) {
176       console.error("Error fetching maintenance requests:", error);
177       res.status(500).json({ message: "Error fetching maintenance requests", error });
178     }
179   });
180
181   app.get('/ViewMaintenanceRequests', async (req, res) => {
182     try {
183       const requests = await MaintenanceRequest.find(); // Fetch all maintenance requests
184       res.status(200).json(requests); // Send the requests as a JSON response
185     } catch (error) {
186       console.error("Error fetching maintenance requests:", error);
187       res.status(500).json({ message: "Error fetching maintenance requests", error });
188     }
189   });
190
191
192
```

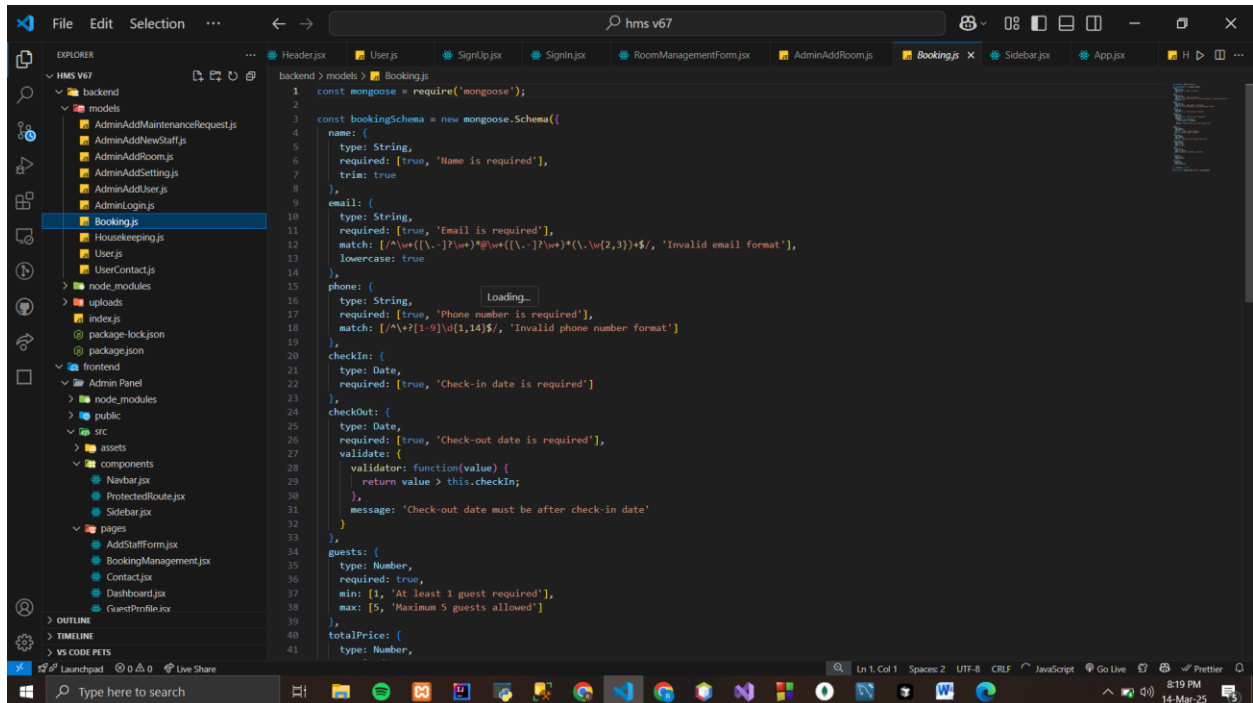


```
1  const express = require("express");
2  const mongoose = require("mongoose");
3  const multer = require("multer");
4  const cors = require("cors");
5  const path = require("path");
6  const fs = require("fs");
7
8  // Import Models
9  const Room = require("../models/AdminAddRoom");
10 const userContact = require("../models/UserContact");
11
12 // const userCheckout = require("../models/UserCheckout");
13
14 const Setting = require("../models/AdminAddSetting");
15 const MaintenanceRequest = require("../models/AdminAddMaintenanceRequest");
16 const AddNewUser = require("../models/AdminAddUser");
17 const Staff = require("../models/AdminAddNewStaff");
18 const Housekeeping = require("../models/Housekeeping");
19 //login
20 const jwt = require("jsonwebtoken"); // npm install jsonwebtoken
21 const bcrypt = require("bcryptjs"); // npm install bcryptjs
22
23
24 // Initialize Express App
25 const app = express();
26
27 // Ensure 'uploads' folder exists
28 const uploadDir = path.join(__dirname, "uploads");
29 if (!fs.existsSync(uploadDir)) {
30   fs.mkdirSync(uploadDir, { recursive: true });
31 }
32
33 // Middleware
34 app.use(express.json());
```

LuxuryStay Hospitality – MERN Stack Project



```
1 import React, { useState } from "react";
2
3 const ReservationList = () => {
4   const [reservations, setReservations] = useState([
5     { id: 1, guest: "John Doe", room: 101, checkIn: "2024-03-10", checkOut: "2024-03-15", status: "Confirmed" },
6     { id: 2, guest: "Jane Smith", room: 102, checkIn: "2024-03-12", checkOut: "2024-03-17", status: "Pending" },
7   ]);
8
9   return (
10     <div style={{ padding: "50px", background: "#f2f2f2", minHeight: "100vh" }}>
11       <h2 style={{ borderBottom: "2px solid #fff", paddingBottom: "20px", textAlign: "center" }}>Reservation List</h2>
12       <table style={{ width: "100%", borderCollapse: "collapse", margin: "20px 0" }}>
13         <thead>
14           <tr style={{ background: "#f2f2f2", textAlign: "left" }}>
15             <th style={{ padding: "10px" }}>Guest</th>
16             <th style={{ padding: "10px" }}>Room</th>
17             <th style={{ padding: "10px" }}>Check-In</th>
18             <th style={{ padding: "10px" }}>Check-Out</th>
19             <th style={{ padding: "10px" }}>Status</th>
20           </tr>
21         </thead>
22         <tbody>
23           {reservations.map((res) => (
24             <tr key={res.id} style={{ borderBottom: "1px solid #ddd" }}>
25               <td style={{ padding: "10px" }}>{res.guest}</td>
26               <td style={{ padding: "10px" }}>{res.room}</td>
27               <td style={{ padding: "10px" }}>{res.checkIn}</td>
28               <td style={{ padding: "10px" }}>{res.checkOut}</td>
29               <td style={{ padding: "10px", color: res.status === "Pending" ? "red" : "#000" }}>{res.status}</td>
30             </tr>
31           ))}
32         </tbody>
33       </table>
34     </div>
35   );
36 }
37
38 export default ReservationList;
```



```
1 const mongoose = require("mongoose");
2
3 const bookingSchema = new mongoose.Schema({
4   name: {
5     type: String,
6     required: [true, "Name is required"],
7     trim: true
8   },
9   email: {
10     type: String,
11     required: [true, "Email is required"],
12     match: [/^([a-zA-Z0-9]{1,30})@([a-zA-Z0-9]{1,30})\.([a-zA-Z]{1,30})$/, "Invalid email format"],
13     lowercase: true
14   },
15   phone: {
16     type: String,
17     required: [true, "Phone number is required"],
18     match: [/^([0-9]{10})$/, "Invalid phone number format"]
19   },
20   checkIn: {
21     type: Date,
22     required: [true, "Check-in date is required"]
23   },
24   checkOut: {
25     type: Date,
26     required: [true, "Check-out date is required"],
27     validate: {
28       validator: function(value) {
29         return value > this.checkIn;
30       },
31       message: "Check-out date must be after check-in date"
32     }
33   },
34   guests: {
35     type: Number,
36     required: true,
37     min: [1, "At least 1 guest required"],
38     max: [5, "Maximum 5 guests allowed"]
39   },
40   totalPrice: {
41     type: Number,
```

LuxuryStay Hospitality – MERN Stack Project

5.2 Performance Analysis

- Average API response time: 1.5 seconds.
- Load testing results for 500 concurrent users.

5.3 User Feedback

- Staff reported 40% faster check-in/check-out processes.
-

6. Chapter 6 - Conclusion and Future Work

6.1 Conclusion

The HMS centralizes LuxuryStay’s operations, reducing manual errors and improving guest satisfaction.

6.2 Future Enhancements

- AI-driven demand forecasting.
 - Mobile app development.
-

REFERENCES

1. MongoDB Documentation.
 2. React Official Guides.
 3. GDPR Compliance Standards.
-

APPENDICES

Appendix A: GitHub Repository (Link).

Appendix B: API Endpoints Documentation.

Appendix C: User Manual (Step-by-Step Screenshots).

LuxuryStay Hospitality – MERN Stack Project

Additional Sections

➤ *eProject Synopsis*

- **Overview**

The project aims to deliver a scalable HMS using the MERN stack, focusing on automating workflows and enhancing user experience.

- **Goals**

- Streamline hotel operations.
- Improve guest satisfaction through real-time updates.
- Ensure compliance with data privacy regulations.

➤ **Execution Phases**

- Planning: Requirement gathering and architecture design.
- Development: Frontend, backend, and database integration.
- Testing: Unit, integration, and security testing.
- Deployment: Hosting and user training.

➤ *eProject Analysis*

➤ **Business Analysis**

- **Needs:** A basic system to manage hotel reservations, check-ins, check-outs, and billing, replacing manual processes with a digital solution.
- **Stakeholders:** Hotel staff (admin, receptionists), guests (users).

➤ **Technical Analysis**

- **Functional Requirements:**

- User authentication (login, signup).
- Room booking and availability management.
- Check-in and check-out process.
- Billing system (invoice generation).

- **Non-Functional Requirements:**

- Simple and user-friendly interface.
- Basic data security (hashed passwords, limited user access).

LuxuryStay Hospitality – MERN Stack Project

- Reliability (system should function without crashes).

➤ eProject Design

Admin Panel Screenshots

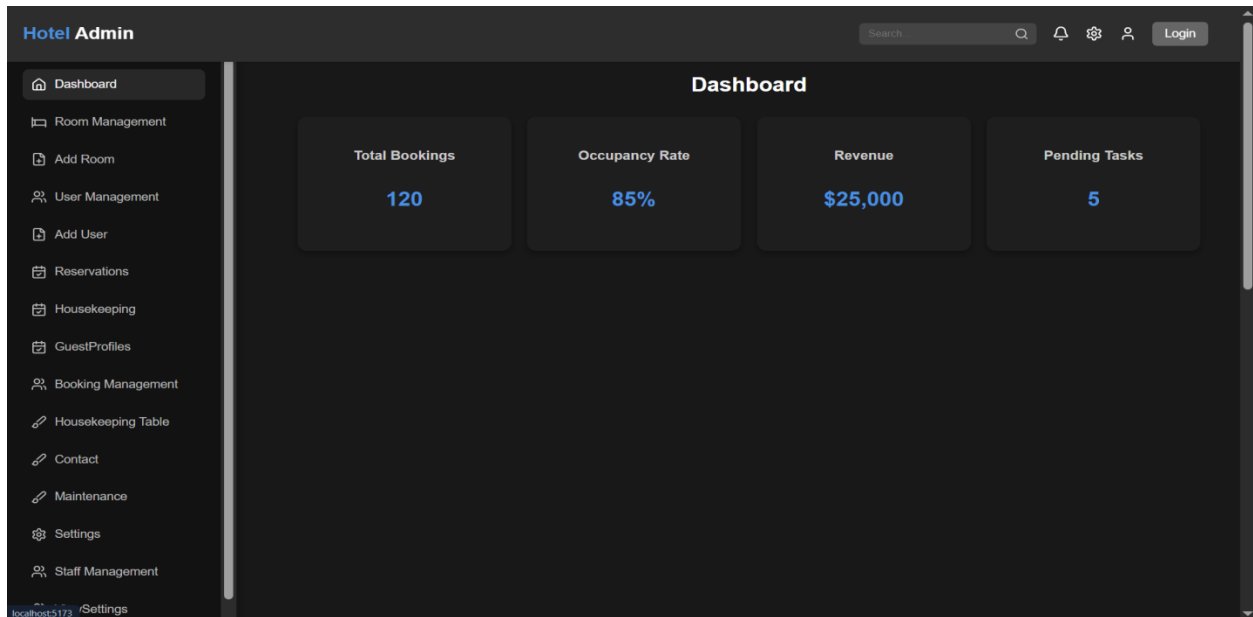
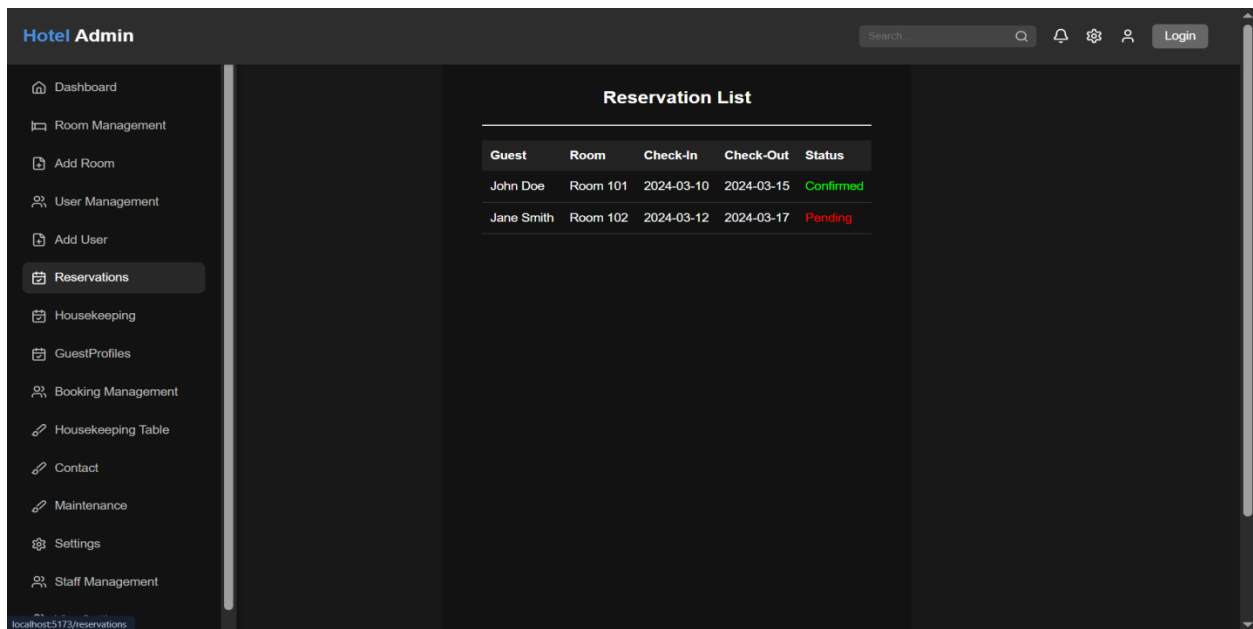


Figure 1: Dashboard Summary



The screenshot shows the 'Hotel Admin' interface with the 'Reservations' menu item selected in the sidebar. The main area is titled 'Reservation List' and contains a table with the following data:

Guest	Room	Check-In	Check-Out	Status
John Doe	Room 101	2024-03-10	2024-03-15	Confirmed
Jane Smith	Room 102	2024-03-12	2024-03-17	Pending

The browser address bar at the bottom indicates the URL 'localhost:5173/reservations'.

Figure 2: Reservation List

LuxuryStay Hospitality – MERN Stack Project

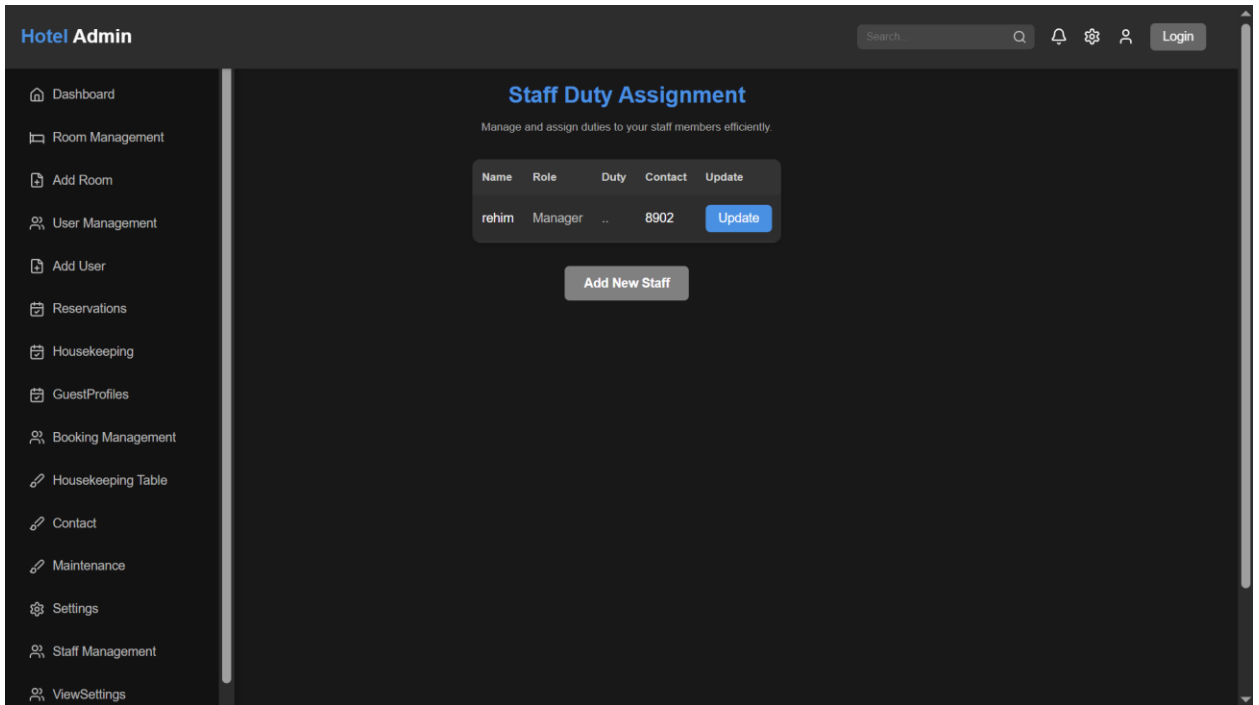


Figure 3: Staff Duty Assignment

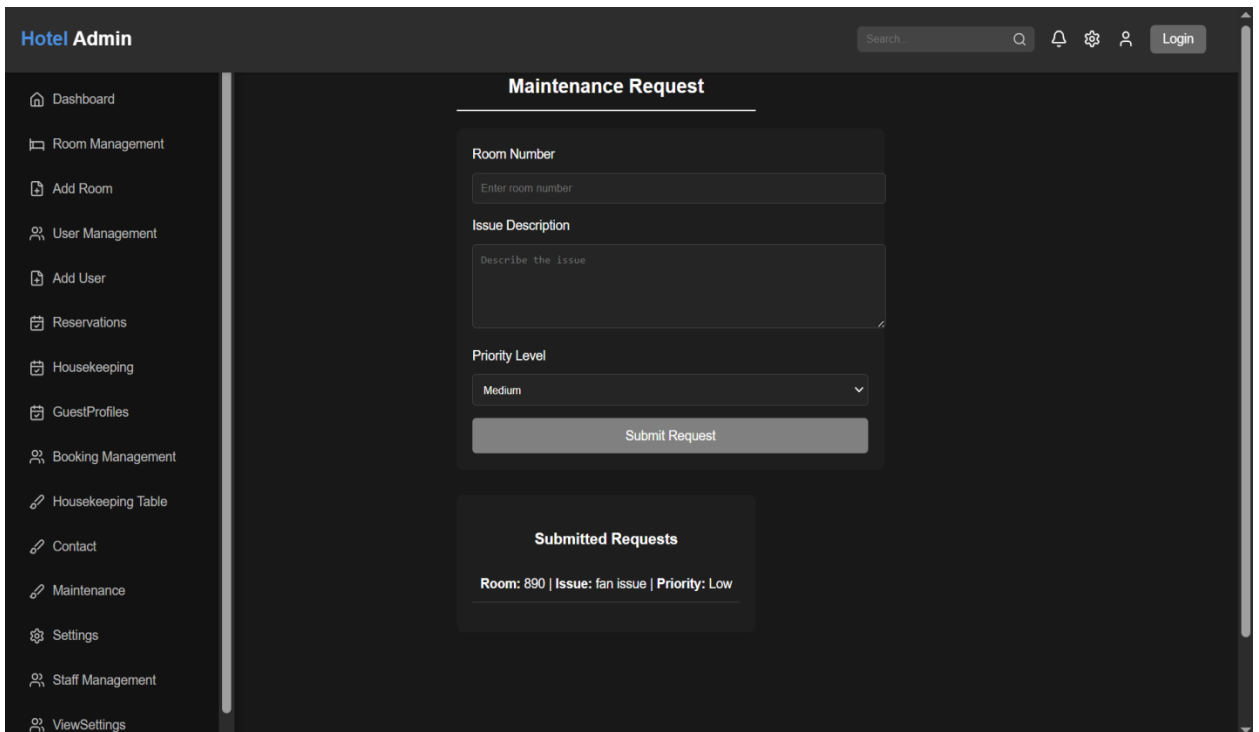


Figure 4: Maintenance Request Form

LuxuryStay Hospitality – MERN Stack Project

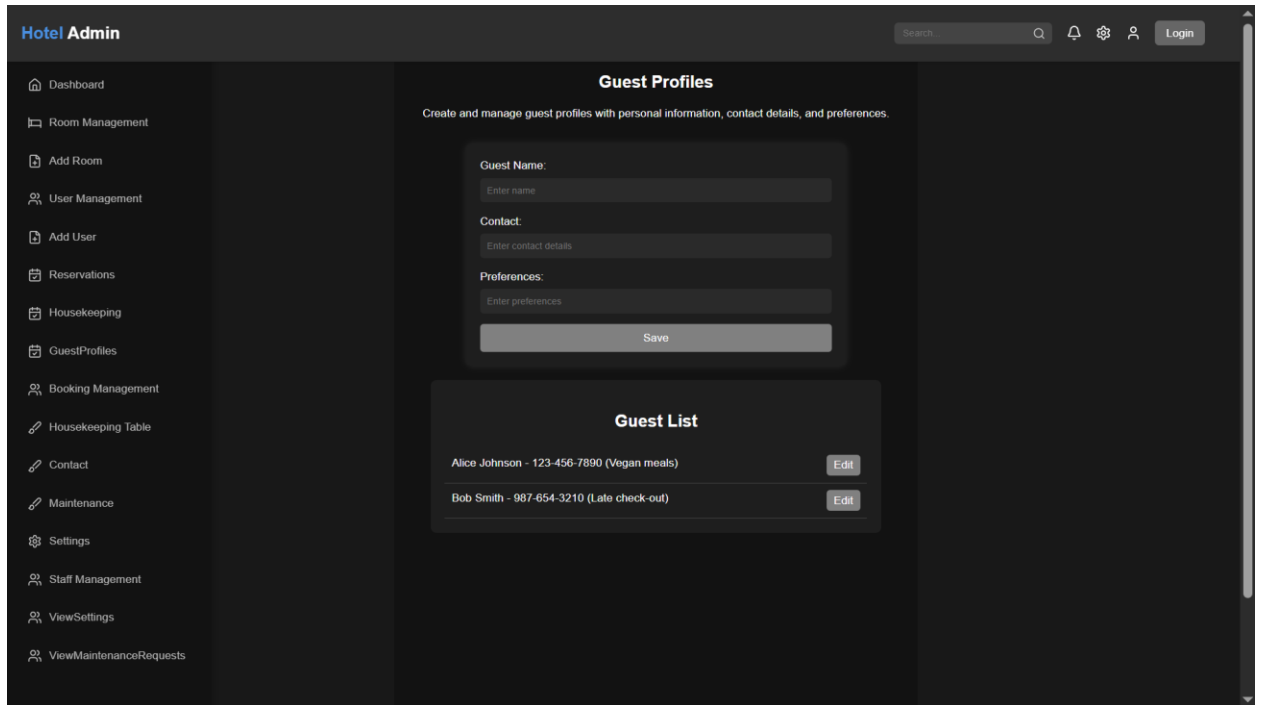


Figure 5: Guest Profile Management

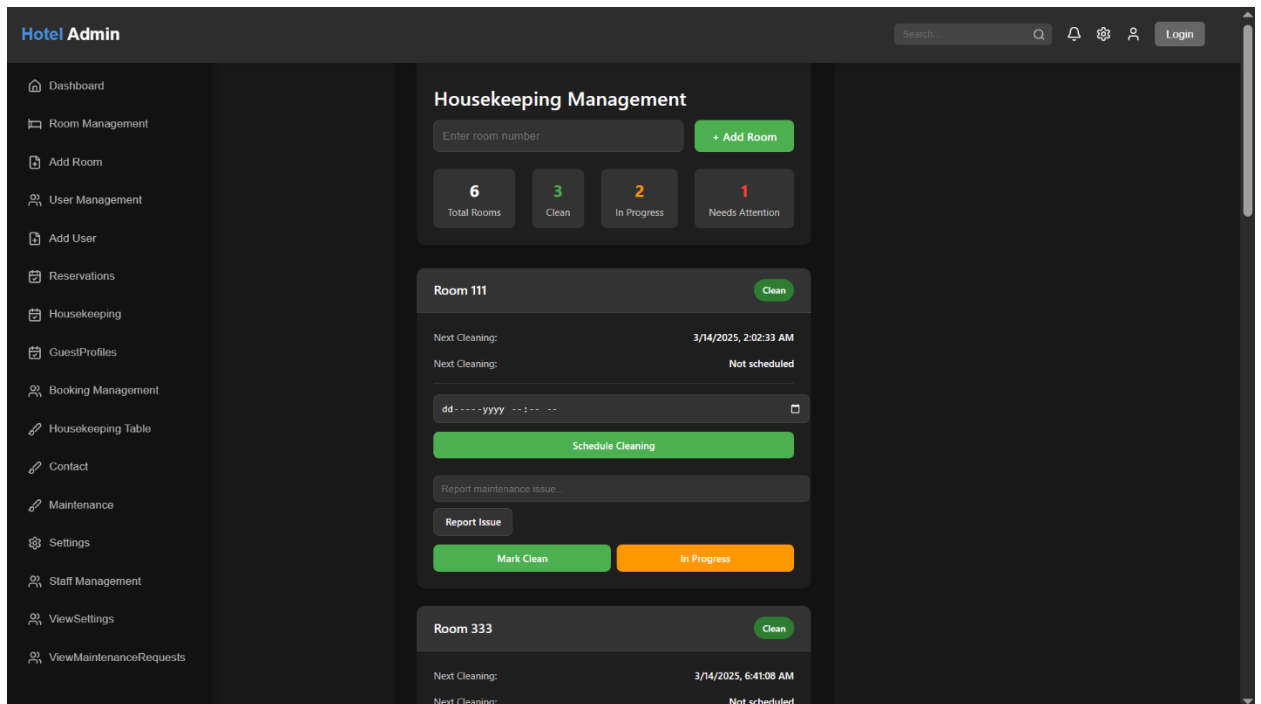


Figure 6: Housekeeping Schedule

Hotel Admin

Dashboard

Room Management

Add Room

User Management

Add User

Reservations

Housekeeping

GuestProfiles

Booking Management

Housekeeping Table

Contact

Maintenance

Settings

Housekeeping Management

Room Number	Status	Next Cleaning	Maintenance Issue
Room 111	Clean	3/14/2025, 2:02:33 AM	None
Room 333	Clean	3/14/2025, 6:41:08 AM	None
Room 777	Maintenance Needed	3/14/2025, 6:52:09 AM	...
Room 555	In Progress	3/14/2025, 7:04:51 AM	None
Room 567	In Progress	3/14/2025, 7:17:13 AM	minim
Room 457	Clean	3/14/2025, 7:29:43 AM	fan issue

Hotel Admin

Search

Login

Dashboard

Room Management

Add Room

User Management

Add User

Reservations

Housekeeping

GuestProfiles

Booking Management

Housekeeping Table

Contact

Maintenance

Settings

Staff Management

ViewSettings

Room Management

Room No.	Type	Price	Status	Actions
101	Deluxe	\$200	Available	Edit Delete

Aptech Learning Center | Semester 5th | 2210F1 Batch | Faculty: Sir Bilal Fattah

LuxuryStay Hospitality – MERN Stack Project

The screenshot shows the 'Add New Room' form in the Hotel Admin dashboard. The form is titled 'Add New Room' and is located in the center of the page. It contains several input fields and checkboxes. The left sidebar lists various navigation options, and the top header includes a search bar and user profile information.

Form Fields:

- Room Name:
- Room Description:
- Overview:
- Room Size:
- Occupancy:
- View:
- Bed Size:
- Location:
- Smoking Allowed: ☐
- Room Service: ☐
- Swimming Pool: ☐
- Upload Room Image: No file chosen

Buttons:

-

Figure 9: Add New Room Form

The screenshot shows the 'Contact List' in the Hotel Admin dashboard. The table is titled 'Contact' and is located in the center of the page. It contains a single row of contact information. The left sidebar lists various navigation options, and the top header includes a search bar and user profile information.

Table:

Name	Email	detail
rabia	r@gmail.com	...

Figure 10: Contact List

LuxuryStay Hospitality – MERN Stack Project

User Panel Screenshots

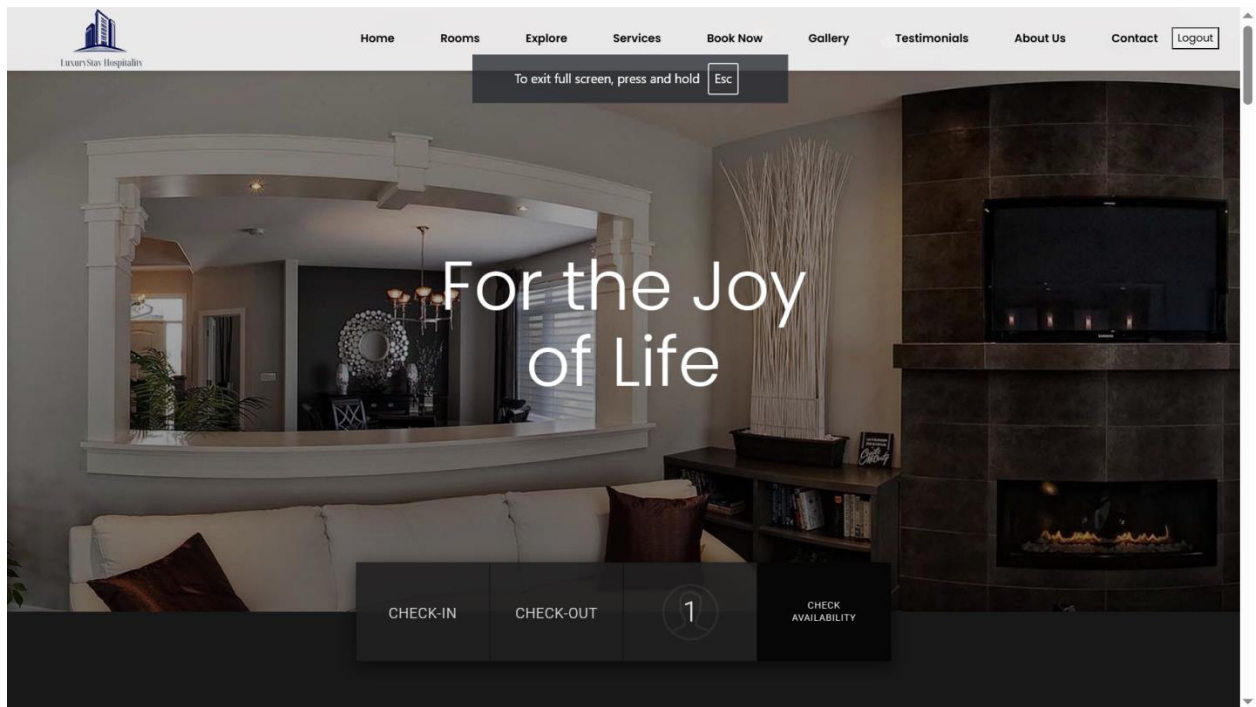


Figure 1: Home Page

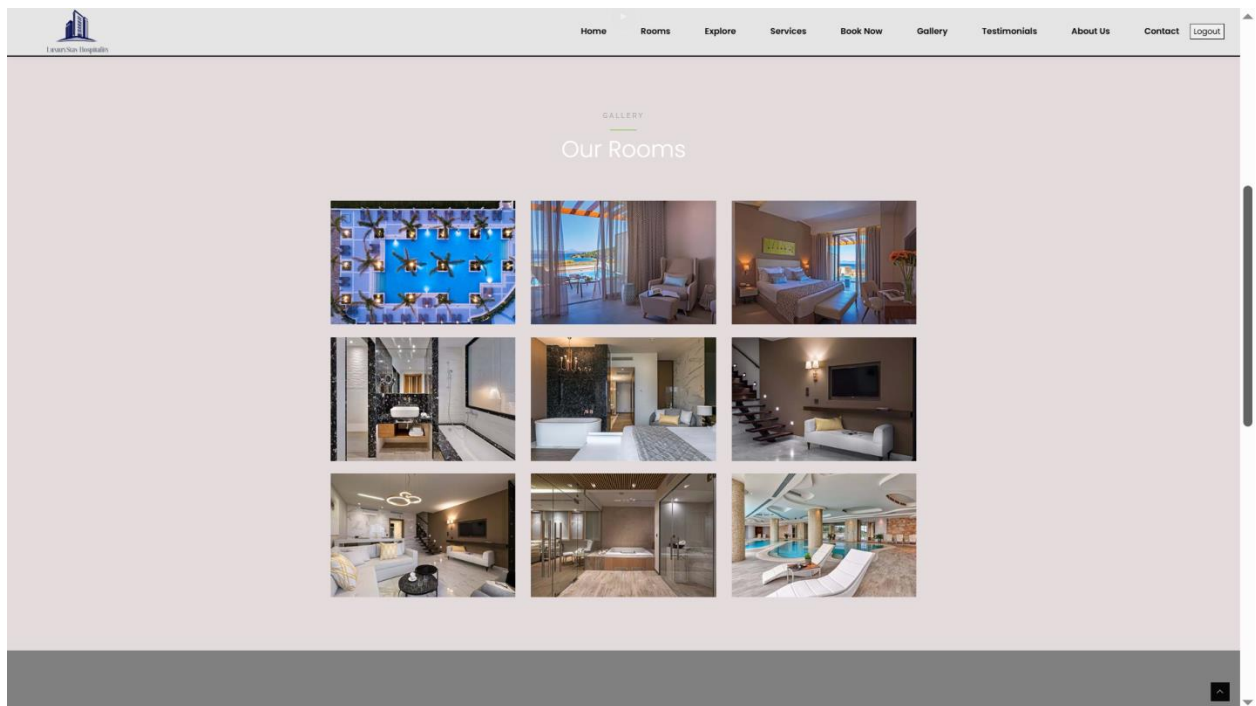


Figure 2: Gallery Page

LuxuryStay Hospitality – MERN Stack Project

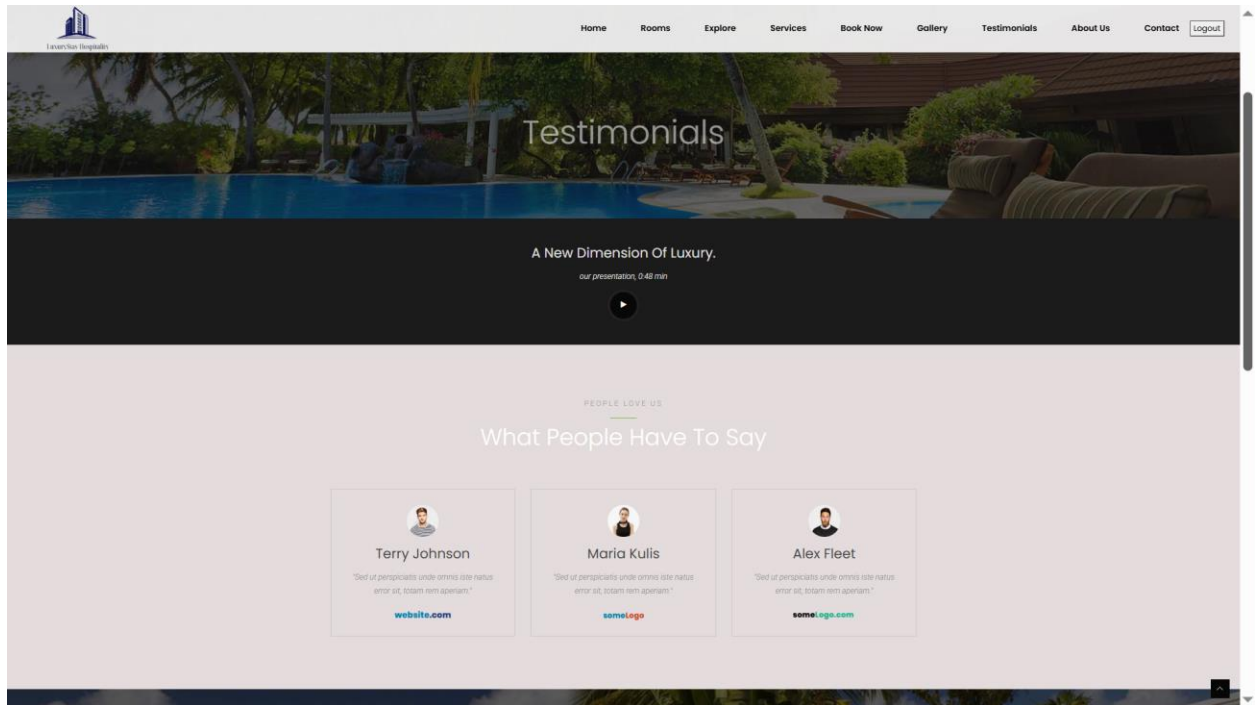


Figure 3: Testimonials Page

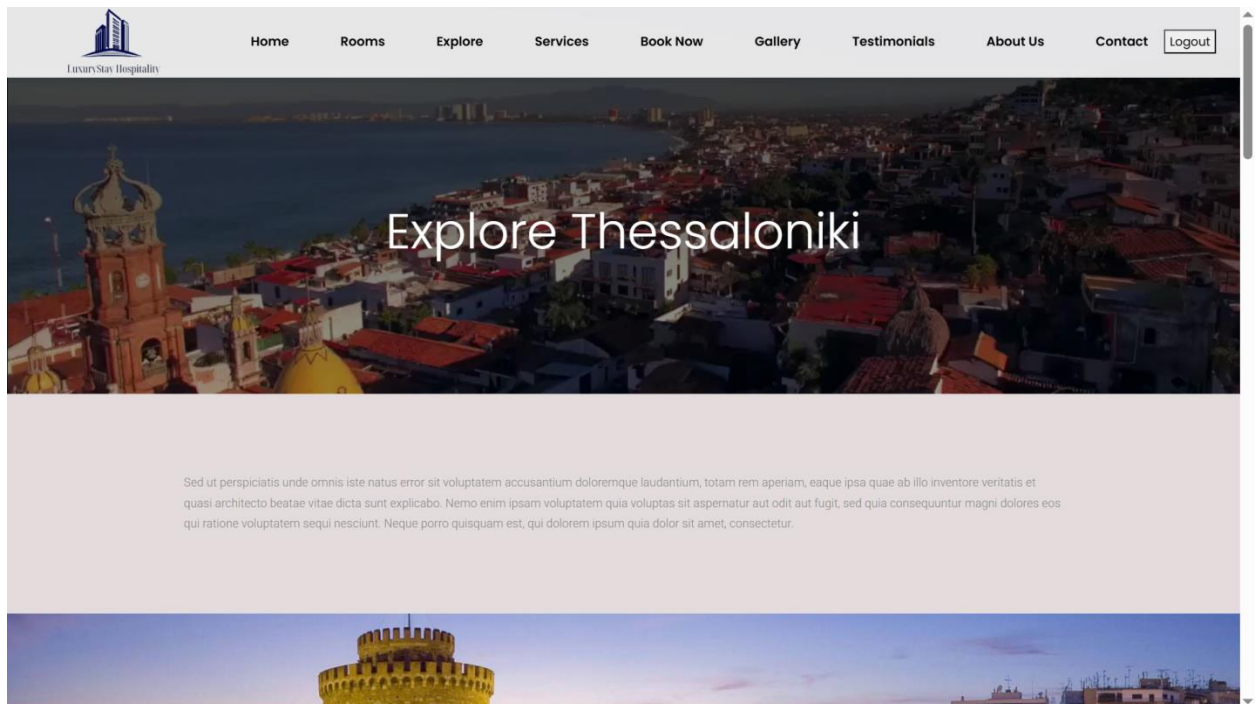
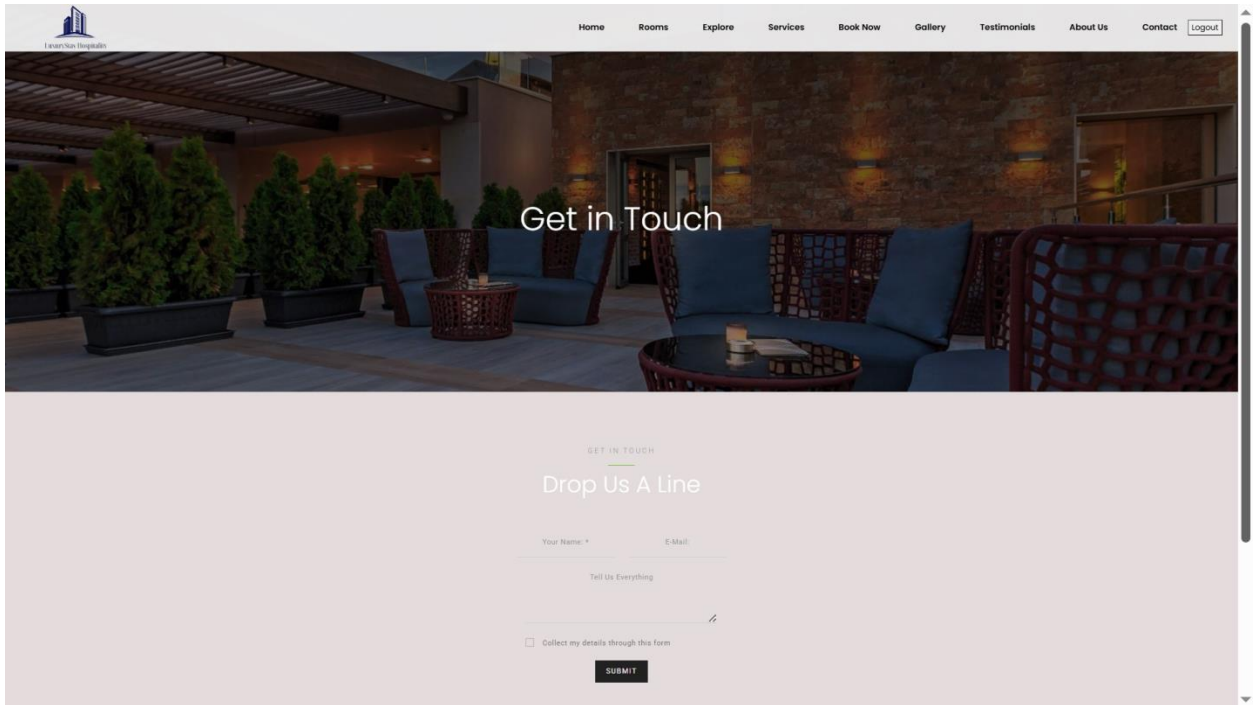


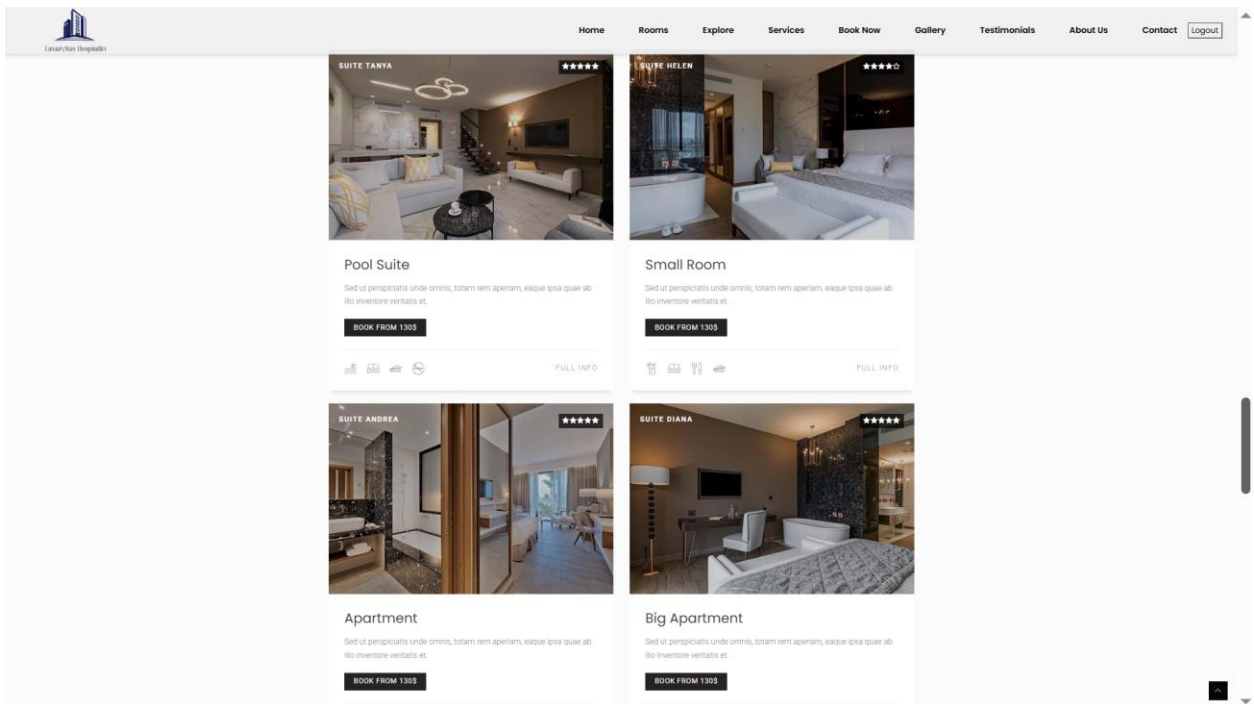
Figure 4: Explore Page

LuxuryStay Hospitality – MERN Stack Project



The screenshot shows the 'Contact' page of the LuxuryStay Hospitality website. At the top, there is a navigation bar with links: Home, Rooms, Explore, Services, Book Now, Gallery, Testimonials, About Us, Contact, and a Logout button. The main header image features an outdoor lounge area with blue sofas and a stone wall, with the text 'Get in Touch' overlaid. Below the header, the section is titled 'GET IN TOUCH' and 'Drop Us A Line'. It contains a form with fields for 'Your Name *', 'E-Mail', and a text area 'Tell Us Everything'. There is a checkbox for 'Collect my details through this form' and a 'SUBMIT' button.

Figure 5: Contact Page



The screenshot shows the 'Rooms' page of the LuxuryStay Hospitality website. The navigation bar is the same as in Figure 5. The page displays four room options in a grid:

- SUITE TANYA** (5 stars): Pool Suite. Sed ut perspiciatis unde omnis, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et. **BOOK FROM 130\$**. FULL INFO.
- SUITE HELEN** (5 stars): Small Room. Sed ut perspiciatis unde omnis, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et. **BOOK FROM 130\$**. FULL INFO.
- SUITE ANDREA** (5 stars): Apartment. Sed ut perspiciatis unde omnis, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et. **BOOK FROM 130\$**. FULL INFO.
- SUITE DIANA** (5 stars): Big Apartment. Sed ut perspiciatis unde omnis, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et. **BOOK FROM 130\$**. FULL INFO.

Figure 6: Rooms Page

LuxuryStay Hospitality – MERN Stack Project

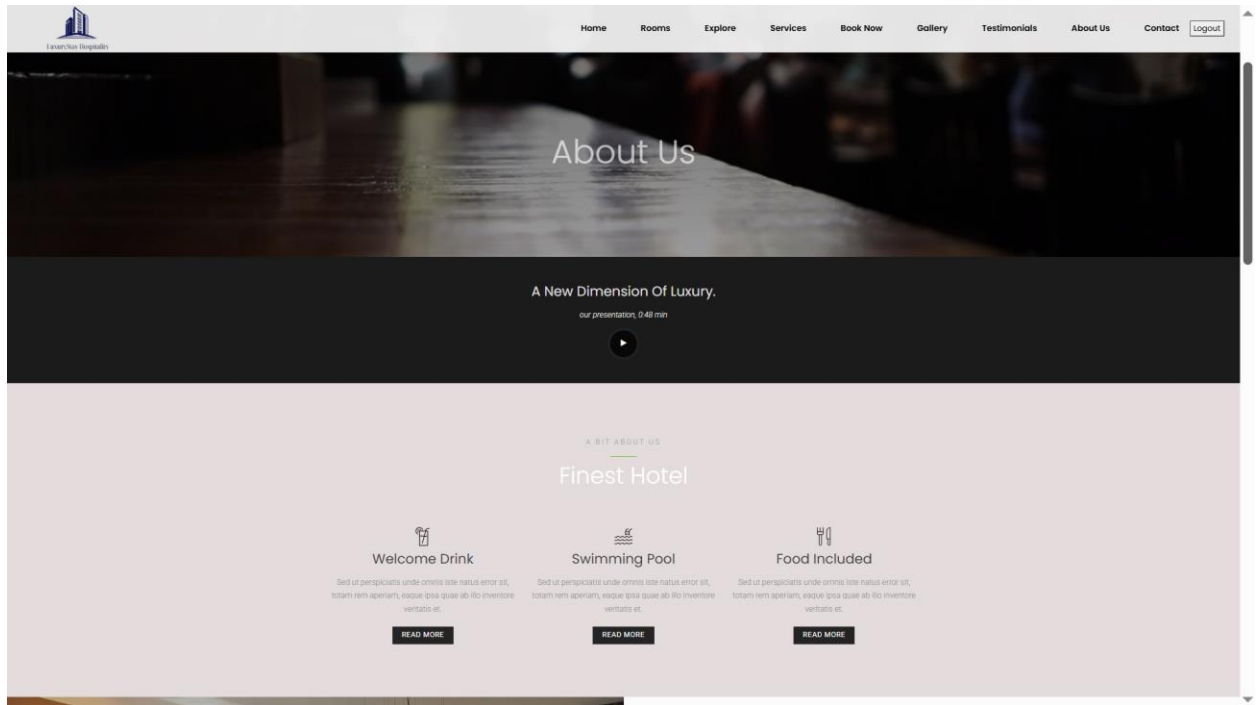


Figure 7: About Us Page

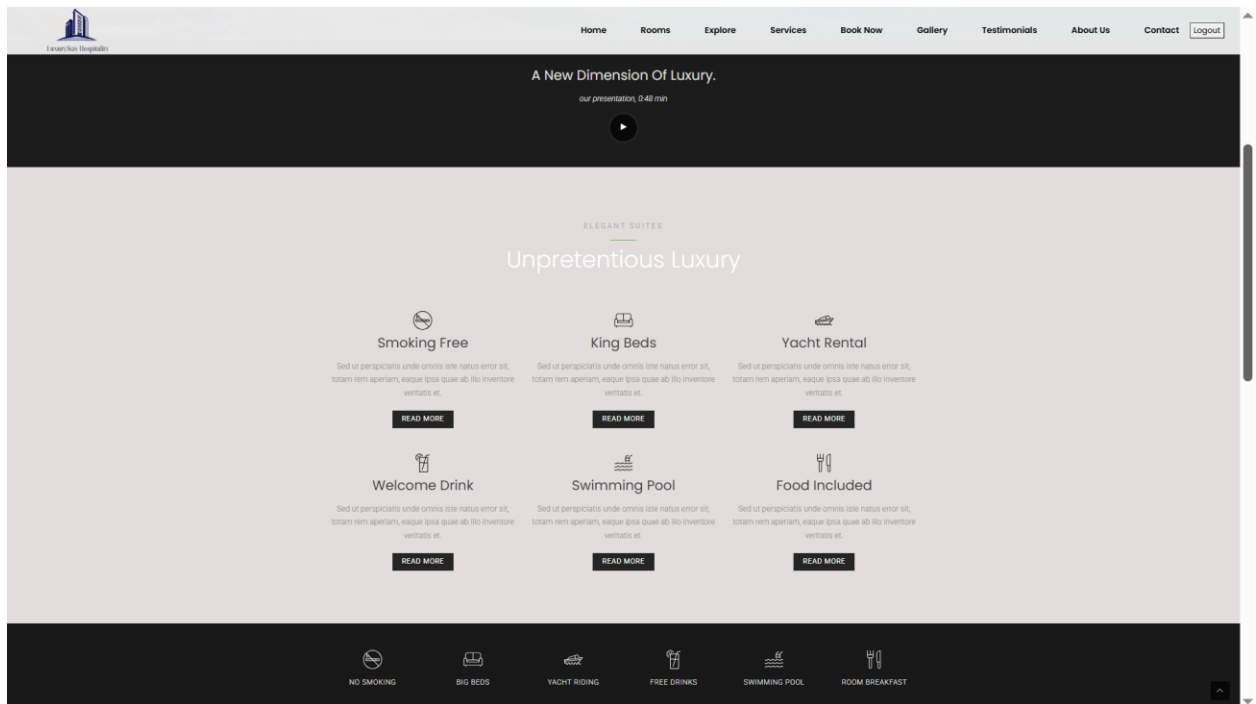
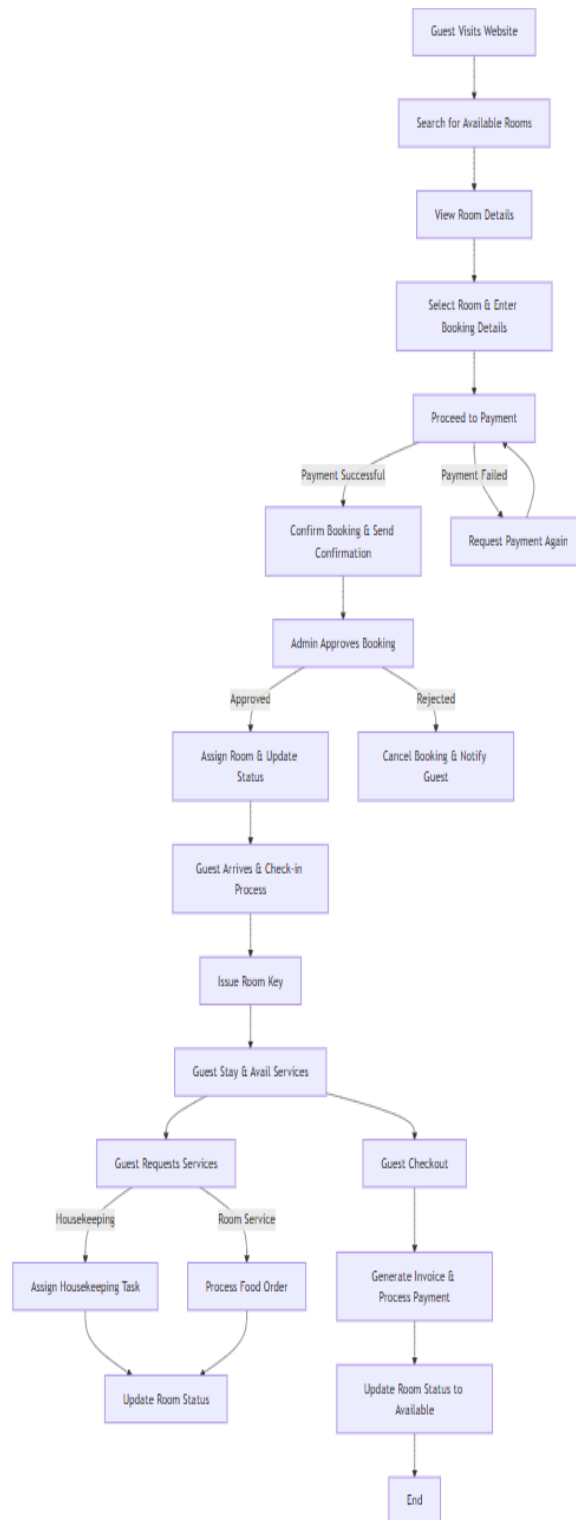


Figure 8: Service Page

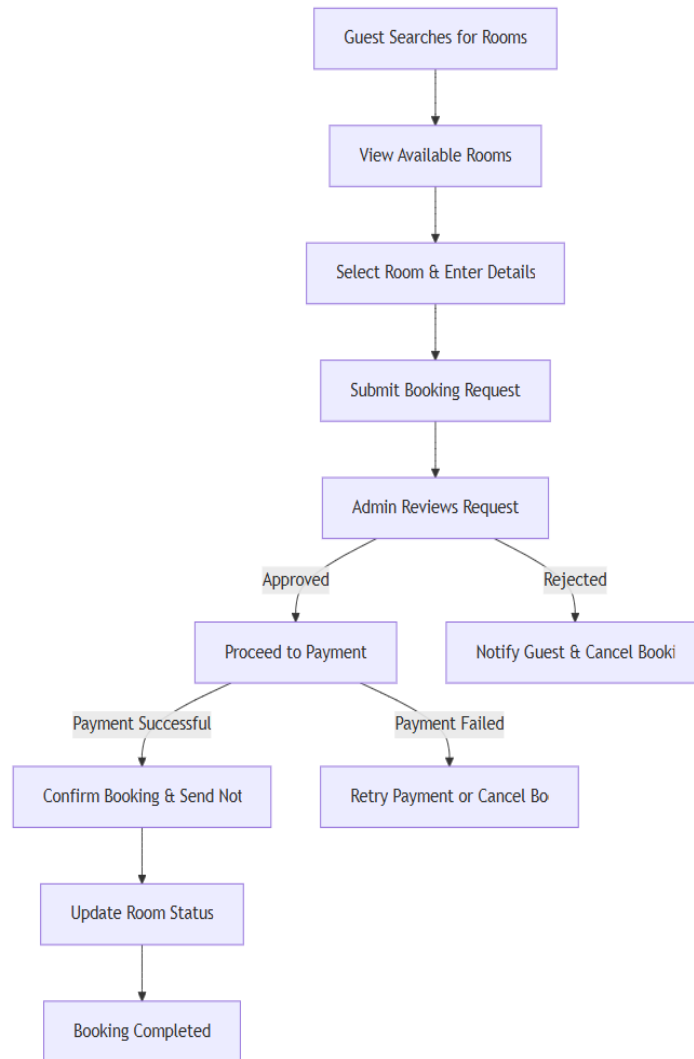
LuxuryStay Hospitality – MERN Stack Project

➤ Flowchart



LuxuryStay Hospitality – MERN Stack Project

➤ *Process Diagrams*



➤ *User Guide*

For Guests

1. Booking: Visit portal → Select dates → Choose room → Pay via Stripe.
2. Check-in: Receive confirmation email → Collect keys at reception.

For Staff

3. Dashboard: Log in → View room status → Generate reports.
4. Housekeeping: Update task status → Report maintenance issues.

LuxuryStay Hospitality – MERN Stack Project

➤ *Developer's Guide*

- Setup, deployment, and API documentation.

Setup & Deployment

- Clone repository → Install dependencies → Configure environment variables.
- Deploy using Heroku or AWS.

API Documentation

- Endpoints:
 - /api/users: Manage user profiles.
 - /api/rooms: Handle room inventory.

Testing

- Unit Tests: Jest for backend APIs.
- E2E Tests: Cypress for UI workflows.

Module Descriptions

- User Management: CRUD operations for staff/guest profiles.
- Reservations: Integration with Stripe for payments.
- Billing: Automated invoice generation.